

Bash trap: A Practical Guide

Signals, cleanup, errors, and exam-ready examples

1. What is trap?

The Bash **trap** builtin tells a script to run a command or function when a specified signal or shell event occurs. It is commonly used to respond to Ctrl+C, clean up temporary files, and handle errors.

```
trap 'command' SIGNAL
```

The action is quoted so Bash stores it and executes it later when the event occurs.

2. Common events

Event	Meaning	Typical use
SIGINT	Interrupt, usually Ctrl+C	Stop safely
SIGTERM	Termination request	Clean shutdown
EXIT	Shell is exiting	Delete temporary files
ERR	A command failed	Show an error
DEBUG	Before many commands	Debugging
RETURN	Function or sourced script returns	Tracing functions

3. Handling Ctrl+C

Ctrl+C normally sends SIGINT. A trap can display a message and then exit with a status code.

```
#!/bin/bash

trap 'echo Interrupted by Ctrl+C; exit 1' SIGINT

echo Running... Press Ctrl+C
while true
do
    sleep 2
done
```

4. Cleanup with EXIT

Use EXIT when something must be cleaned up whenever the script ends. The official Bash manual states that a trap on EXIT is executed before the shell terminates.

```
#!/bin/bash

file="temp.txt"
touch "$file"

cleanup() {
    rm -f "$file"
    echo "Temporary file removed"
}

trap cleanup EXIT

echo "Working..."
sleep 2
```

5. Cleanup after Ctrl+C too

```
#!/bin/bash
```

```
temp_file="temp.txt"
touch "$temp_file"

cleanup() {
    rm -f "$temp_file"
    echo "Cleanup completed"
}

trap cleanup EXIT
trap 'echo Interrupted; exit 1' SIGINT

sleep 20
```

Why this works:

The SIGINT trap exits the script. Exiting causes the EXIT trap to run, which calls cleanup and removes the file.

6. Handling errors with ERR

```
#!/bin/bash

trap 'echo A command failed' ERR

cat missing_file.txt
echo Script continues unless you exit
```

To stop after an unhandled command failure:

```
#!/bin/bash
set -e
trap 'echo Error occurred; exiting' ERR

cat missing_file.txt
echo This will not run
```

Important: ERR does not trigger in every context. For example, failures used directly as an if condition are treated specially. Do not rely on ERR alone for every kind of validation.

7. Remove or inspect traps

```
trap - SIGINT      # remove the SIGINT trap
trap -p           # display installed traps
trap -l           # list signal names and numbers
```

8. Exam syntax checklist

Remember these points:

- Use quotes around the action: `trap 'commands' SIGNAL`.
- Place trap near the beginning of the script.
- Use a function when cleanup has multiple commands.
- Use `rm -f` for safe removal of a temporary file.
- Use `exit 1` for abnormal termination and `exit 0` for success.
- Test file existence or permissions before reading or changing files.

9. Practice question

Write a script that creates a temporary file, displays a message, and removes the file both when the script finishes normally and when the user presses Ctrl+C.

```
#!/bin/bash

temp="temp.txt"
```

```
touch "$temp"

cleanup() {
    rm -f "$temp"
    echo "Deleted $temp"
}

trap cleanup EXIT
trap 'echo Stopped; exit 1' SIGINT

echo Temporary file created
sleep 10
```

Reference

GNU Bash Reference Manual, sections on the trap builtin, signals, exit status, and shell scripts. Download the complete manual before your test if internet access will be unavailable.